

Computación de Alto Rendimiento

Grado en Ingeniería Informática

PRÁCTICA DE AULA

Práctica 2 · Tema 2

Procesos, hilos y carreras de datos: coste y corrección

Práctica · Núcleo de 2 h en clase y trabajo autónomo · Grupos de 2-3

Docentes: Elam Uceda Herrero · Javier Vázquez Pereda



Ficha de la práctica

Esta es la práctica del Tema 2, y reúne en una sola las dos que había antes. Se hace **después de la teoría de los apartados 2.1 y 2.2**: ya sabes que un proceso cuesta más que un hilo, qué es una condición de carrera y con qué se arregla. Hoy lo compruebas con tus propios números. **No necesitas nada que no esté en el Tema 2**: el reloj con el que se mide viene incluido, los programas del núcleo se dan casi completos y los del trabajo autónomo traen su pieza clave y las pistas que hacen falta.

APARTADO	DETALLE
Asignatura	Computación de Alto Rendimiento · 4.º de Ingeniería Informática · Tema 2, <i>Programación concurrente</i>
Organización	Núcleo en clase : una sesión de 2 horas, ejercicios 1, 2 y 3. Trabajo autónomo obligatorio : unas 3 horas fuera del aula, ejercicios 4 y 5. Los cinco ejercicios puntúan
Modalidad	Grupos de 2 o 3 personas. Un solo juego de programas por grupo, y todos los nombres en la entrega
Entorno	<code>gcc</code> con <code>-lpthread</code> . Sirve Windows (el gcc de CLion o de MinGW), Linux o WSL
Requisito previo	Haber estudiado los apartados 2.1 y 2.2 , y llegar con el entorno comprobado: <code>comprueba.c</code> tiene que decir ENTORNO LISTO antes de la sesión
Qué se entrega	Un único PDF por equipo , con el código pegado dentro, las tablas de medidas, las gráficas que se piden, las conclusiones y la declaración del trabajo y del uso de IA. No se entregan ficheros de código sueltos ni comprimidos
Peso	Los 5 ejercicios suman 10 puntos : 6 el núcleo en clase y 4 el trabajo autónomo

CÓMO REPARTIR LAS DOS HORAS DE CLASE

Es una estimación, no una norma. Si vas muy por detrás de este reparto, avisa en clase.

Leer el guion, 10 min · Ejercicio 1, 25 min · Ejercicio 2, 25 min · Ejercicio 3, 45 min · conclusiones y dudas, 15 min. Lo que no dé tiempo a redactar en clase se completa en casa, junto con el trabajo autónomo.

Objetivo

Al terminar tienes que poder responder con números, y no con opiniones, a estas preguntas:

- ¿Cuánto cuesta crear un hilo? ¿Y un proceso? ¿Por qué la respuesta cambia tanto de Windows a Linux?
- ¿Una condición de carrera falla siempre? Si no, ¿de qué depende que se vea, y qué significa eso para probar un programa?
- ¿Cuánto cuesta cada forma correcta de arreglarla —cerrojo, operación atómica o acumulador privado—, y por qué el cerrojo empeora al añadir hilos?
- ¿Por qué se cuelga un programa con dos cerrojos, y por qué se arregla pidiéndolos siempre en el mismo orden?

No hay teoría nueva: todo está en los apartados 2.1 y 2.2, y cada ejercicio dice de qué apartado sale.

Antes de la sesión: el entorno, comprobado

La sesión de clase no tiene tiempo para instalar nada. **Llega con esto hecho.** Si el comprobador no dice ENTORNO LISTO, avisa al profesor antes del día de la práctica, no durante.

El reloj: `crono.h`

Para medir lo que cuesta crear un hilo hace falta un reloj que distinga microsegundos. `time()` solo cuenta segundos, y `clock()` tampoco sirve: en Windows devuelve tiempo de pared y en Linux tiempo de procesador, así que las dos plataformas te dirían cosas distintas. Guarda este fichero con el nombre `crono.h` en la misma carpeta que tus programas; todos los de esta práctica lo incluyen:

```
/* crono.h - un reloj de pared fino y portable.
   time() solo cuenta segundos y clock() no mide lo mismo en Windows que en
   Linux, así que ninguno sirve para medir microsegundos.
   Aquí se usa QueryPerformanceCounter (100 ns) en Windows y
   clock_gettime(CLOCK_MONOTONIC) en Linux. Devuelve segundos. */
#ifndef CRONO_H
#define CRONO_H

#ifdef _WIN32
#include <windows.h>
static double reloj(void)
{
    LARGE_INTEGER f, t;
    QueryPerformanceFrequency(&f);
    ❶ QueryPerformanceCounter(&t);
    return (double)t.QuadPart / (double)f.QuadPart;
}
#else
#include <time.h>
static double reloj(void)
{
    struct timespec t;
    ❷ clock_gettime(CLOCK_MONOTONIC, &t);
    return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
}
#endif
#endif
```

❶ En Windows se usa el contador de alta resolución del sistema: en el portátil con el que se ha preparado esta práctica, su salto mínimo es de 100 ns. ❷ En Linux, el reloj monótono, que no retrocede aunque alguien cambie la hora del sistema. Las dos ramas devuelven **segundos** en un `double`.

El comprobador: `comprueba.c`

Crea cuatro hilos, espera a los cuatro y mide el salto más pequeño que da el reloj. Compíllalo y ejecútalo en el ordenador con el que vayas a trabajar:

```
/* comprueba.c - ejecutalo ANTES de la sesion. Si no dice ENTORNO LISTO,
   la practica no se puede hacer en clase. */
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>
#include "crono.h"
```

```

static atomic_long terminados;

static void *trabajador(void *arg)
{
    (void)arg;
    ❶ atomic_fetch_add(&terminados, 1);
    return NULL;
}

int main(void)
{
    pthread_t h[4];
    double t0 = reloj();
    for (int i = 0; i < 4; i++) pthread_create(&h[i], NULL, trabajador, NULL);
    for (int i = 0; i < 4; i++) pthread_join(h[i], NULL);
    double t = reloj() - t0;

    double paso = 1.0; /* el menor salto que da el reloj */
    for (int i = 0; i < 1000; i++) {
        double a = reloj(), b;
        ❷ do b = reloj(); while (b == a);
        if (b - a < paso) paso = b - a;
    }
    long n = atomic_load(&terminados);
    printf("hilos terminados      : %ld de 4\n", n);
    printf("crear y esperar 4 hilos : %.3f ms\n", t * 1e3);
    printf("resolucion del reloj      : %.0f ns\n", paso * 1e9);
    printf("%s\n", (n == 4 && paso < 1e-6) ? "ENTORNO LISTO"
        : "ALGO FALLA: avisa antes de la sesion");

    return 0;
}

```

❶ Cada hilo suma uno con una operación atómica (apartado 2.2.5): si el programa dice «4 de 4», los hilos se crean, corren y terminan. ❷ Este bucle pide la hora hasta que cambia; la diferencia es el salto mínimo del reloj. Si pasa de un microsegundo, el reloj no sirve para esta práctica y el comprobador lo dice.

```

$ gcc -O2 -Wall -Wextra -o comprueba comprueba.c -lpthread
$ ./comprueba
hilos terminados      : 4 de 4
crear y esperar 4 hilos : 0.509 ms
resolucion del reloj   : 100 ns
ENTORNO LISTO

```

Esa es la salida en Windows 11. En el mismo portátil con Linux (WSL 2, Ubuntu 24.04) sale `0.435 ms` y `19 ns`. Tus cifras serán otras; lo que importa es la última línea.

SI TRABAJAS EN WINDOWS El ejercicio 1 lanza un proceso que no hace nada. Prepáralo también antes de la sesión, en la misma carpeta: un fichero `nada.c` que contenga solo `int main(void) { return 0; }`, compilado con `gcc -o nada.exe nada.c`. En Linux no hace falta.

Antes de empezar

LA REGLA DE ORO DE ESTA PRÁCTICA Una demostración de carrera que no falla es peor que no tenerla.

Con `-O2`, el compilador guarda las variables en registros siempre que puede, y un registro es privado de cada hilo: el error sigue ahí, pero no se manifiesta. Es la sorpresa del apartado 2.2.1, donde el mismo programa falla con `-O0` y no con `-O2`. Por eso las variables compartidas de estos programas llevan `volatile`, que obliga a que vivan en memoria. Y cuando un programa incorrecto te dé el resultado bueno, no escribas que funciona: escribe que **esta vez ha habido suerte**.

La chuleta

NECESITAS	SE ESCRIBE ASÍ	APARTADO
Crear y esperar un hilo	<code>pthread_create(&h, NULL, f, arg)</code> · <code>pthread_join(h, NULL)</code>	2.1.1
Crear un proceso (Windows)	<code>CreateProcessA</code> · <code>WaitForSingleObject</code>	2.1.3
Crear un proceso (Linux)	<code>fork()</code> · el padre lo recoge con <code>waitpid()</code>	2.1.3
Cerrojo	<code>pthread_mutex_lock</code> · <code>pthread_mutex_unlock</code>	2.2.2
Suma atómica	<code>#include <stdatomic.h></code> · <code>atomic_long c;</code> · <code>atomic_fetch_add(&c, 1)</code>	2.2.5
Acumulador privado	variable local al hilo + una sola suma final protegida	2.2.11
Evitar el interbloqueo	orden total: pedir siempre los cerrojos por identificador creciente	2.2.9
Reloj fino	<code>crono.h</code> , incluido en esta práctica	—

Las trampas de hoy

Las cuatro están comprobadas en la máquina con la que se ha preparado esta práctica. Ninguna da un mensaje de error, y cada una invalida una medida entera.

Trampa 1: -O2 esconde las carreras y borra lo que no se usa

Si tu contador compartido no falla nunca, comprueba primero si el compilador lo ha metido en un registro: recompila con `-O0` y mira si entonces falla. Y si un bucle calcula algo que nadie mira, el compilador lo elimina y mides el coste de no hacer nada. Por eso el acumulador privado del ejercicio 3 lleva `volatile` a propósito.

Trampa 2: si el trabajo es corto, la carrera no aparece

Con hilos que dan mil vueltas cada uno, la carrera no se manifiesta: cada hilo termina antes de que el siguiente llegue a arrancar, así que no llegan a solaparse. El programa es igual de incorrecto. Para verla hacen falta millones de vueltas, y eso es justo lo que mide el ejercicio 2.

Trampa 3: no estás solo en la máquina

El navegador, el antivirus y el propio sistema compiten contigo. Repite cada medida y quédate con el **mínimo**: la interferencia solo puede hacer que una pasada tarde más, nunca menos, así que la más rápida es la que menos ruido lleva dentro. Y di siempre cuántas repeticiones hay detrás de cada número y en qué sistema se midió.

Trampa 4: long tiene 32 bits en Windows

En Windows, `long` —y con él `atomic_long`— tiene 32 bits y no pasa de dos mil millones. En esta práctica los contadores llegan como mucho a 240 millones y caben, pero si escribes programas con más vueltas, usa `long long` en todo contador que pueda pasar de esa cifra.

Núcleo en clase: ejercicios 1 a 3 (6 puntos)

Los programas se dan **casi completos**, para que el tiempo de clase sea para medir y entender y no para teclear andamiaje. En el ejercicio 1 compilas, mides y explicas; en el 2 haces además un cambio de una palabra; en el 3 escribes tú las dos versiones centrales. Compila siempre con `-O2 -Wall -Wextra`.

SOBRE LOS NÚMEROS DE ESTE GUION Las tablas de referencia se midieron en un AMD Ryzen AI 9 HX 370, de 12 núcleos físicos y 24 procesadores lógicos, con Windows 11 y gcc 13.1. En septiembre de 2026 se volvieron a ejecutar los programas de este guion en la misma máquina, con gcc 14.2 en Windows y gcc 13.3 en Linux (WSL 2, Ubuntu 24.04): compilan sin avisos, dan el valor exacto donde deben y las conclusiones no cambian. Tus cifras serán distintas; lo que se corrige es la forma y la explicación.

Ejercicio 1 (1,5 puntos) — Lo que cuesta nacer

Enunciado. Compila y ejecuta `p2_ej1.c`: mide cuánto cuesta crear una unidad de ejecución que no hace absolutamente nada (apartado 2.1.3). Cuatro medidas, no dos.

- **En tanda:** crear las N y después esperar a las N . Las creaciones se solapan entre los núcleos, así que esto mide **caudal**.
- **De una en una:** crear, esperar, crear la siguiente. Nada se solapa: esto mide **latencia**, que es lo que paga de verdad un programa que lanza trabajadores dentro de un bucle.
- Las dos cosas para hilos y para procesos. El proceso no hace nada —en Windows es `nada.exe`; en Linux, el hijo termina en el acto—, para que todo el tiempo medido sea del sistema operativo y no del programa.
- El programa publica el mínimo **y la media** de cinco repeticiones. Explica en tus conclusiones por qué se usa el mínimo.

Primero, la parte de hilos. Fíjate en la diferencia entre los dos bucles, que es todo el ejercicio:

```
/* p2_ej1.c - lo que cuesta crear un hilo y un proceso (apartado 2.1.3). */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include "crono.h"
#ifdef _WIN32
#include <unistd.h>
#include <sys/wait.h>
#else
#include <unistd.h>
#endif

#define N_HILOS 1000 /* hilos por medida */
#define N_PROC 200 /* procesos por medida */
#define REP 5 /* repeticiones */

1 static void *nada_hilo(void *arg) { (void)arg; return NULL; }

static double hilos_tanda(int n) /* crear los n y despues esperar a los n */
{
    pthread_t *h = malloc(n * sizeof *h);
```

```

double t0 = reloj();
❷ for (int i = 0; i < n; i++) pthread_create(&h[i], NULL, nada_hilo, NULL);
for (int i = 0; i < n; i++) pthread_join(h[i], NULL);
double t = reloj() - t0;
free(h);
return t / n;
}

static double hilos_uno_a_uno(int n) /* crear, esperar, crear el siguiente */
{
    pthread_t h;
    double t0 = reloj();
    for (int i = 0; i < n; i++) {
        ❸ pthread_create(&h, NULL, nada_hilo, NULL);
        pthread_join(h, NULL);
    }
    return (reloj() - t0) / n;
}

```

- ❶ El trabajador devuelve `NULL` inmediatamente. No queremos medir cálculo, queremos medir nacimiento y muerte.
- ❷ En la versión en tanda se crean los N antes de esperar a ninguno, así que las creaciones se pisan unas a otras y aprovechan los 24 procesadores lógicos.
- ❸ En la versión de una en una, cada `pthread_create` tiene que esperar a que el anterior haya muerto. No se solapa nada. Las dos funciones se diferencian **solo** en dónde está el `join`, y verás que dan números muy distintos.

La parte de procesos es la misma idea con otra llamada, y aquí es donde Windows y Linux se separan (apartado 2.1.3). En Windows no existe `fork()`: `CreateProcessA` construye el proceso entero desde cero, carga el ejecutable, resuelve sus bibliotecas y monta su espacio de direcciones. En Linux, `fork()` duplica el proceso actual con copia perezosa. El programa lleva las dos versiones, y el compilador elige la de tu sistema:

```

#ifdef _WIN32
/* Windows no tiene fork(): se lanza nada.exe con CreateProcessA (apartado 2.1.3). */
static void lanza(PROCESS_INFORMATION *pi)
{
    STARTUPINFOA si;
    char linea[] = "nada.exe";
    ZeroMemory(&si, sizeof si);
    si.cb = sizeof si;
    ❶ if (!CreateProcessA(NULL, linea, NULL, NULL, FALSE, CREATE_NO_WINDOW,
                        NULL, NULL, &si, pi)) {
        fprintf(stderr, "no se puede lanzar nada.exe: compilalo primero\n");
        exit(1);
    }
}

static void espera(PROCESS_INFORMATION *pi)
{
    ❷ WaitForSingleObject(pi->hProcess, INFINITE);
    CloseHandle(pi->hProcess);
    CloseHandle(pi->hThread);
}

static double procesos_tanda(int n)
{
    PROCESS_INFORMATION *pi = malloc(n * sizeof *pi);

```

```

    double t0 = reloj();
    for (int i = 0; i < n; i++) lanza(&pi[i]);
    for (int i = 0; i < n; i++) espera(&pi[i]);
    double t = reloj() - t0;
    free(pi);
    return t / n;
}
static double procesos_uno_a_uno(int n)
{
    PROCESS_INFORMATION pi;
    double t0 = reloj();
    for (int i = 0; i < n; i++) { lanza(&pi); espera(&pi); }
    return (reloj() - t0) / n;
}

```

```

#else
/* Linux: fork() duplica el proceso; el hijo termina en el acto con _exit(0)
   y el padre lo recoge con waitpid(). */
static double procesos_tanda(int n)
{
    pid_t *p = malloc(n * sizeof *p);
    double t0 = reloj();
    for (int i = 0; i < n; i++)
        ❸ if ((p[i] = fork()) == 0) _exit(0);
    ❹ for (int i = 0; i < n; i++) waitpid(p[i], NULL, 0);
    double t = reloj() - t0;
    free(p);
    return t / n;
}
static double procesos_uno_a_uno(int n)
{
    double t0 = reloj();
    for (int i = 0; i < n; i++) {
        pid_t p = fork();
        if (p == 0) _exit(0);
        waitpid(p, NULL, 0);
    }
    return (reloj() - t0) / n;
}
#endif

```

- ❶ En Windows se lanza `nada.exe`; si no está en la carpeta desde la que ejecutas, el programa lo dice y termina.
 ❷ Se espera a que acabe y se cierran sus dos manejadores. ❸ En Linux, `fork()` devuelve 0 en el hijo, que termina en el acto con `_exit(0)`. ❹ El padre lo recoge con `waitpid()`, el equivalente de esperar a un hilo con `pthread_join`.

Y el resto: la función que repite cada medida y el `main`. Los cuatro fragmentos, en este orden, forman `p2_ej1.c`:

```

/* Repite REP veces una medida, imprime minimo y media, devuelve el minimo. */
static double mide(const char *nombre, double (*f)(int), int n)
{
    double suma = 0, min = 1e30;
    for (int r = 0; r < REP; r++) {
        double t = f(n);
        suma += t;
        if (t < min) min = t;
    }
}

```

```

}
printf("%-18s: %10.1f us %10.1f us\n", nombre, min * 1e6, suma / REP * 1e6);
return min;
}

int main(void)
{
    printf("%-18s %13s %13s (%d repeticiones)\n", "", "minimo", "media", REP);
    double a = mide("hilo en tanda", hilos_tanda, N_HILOS);
    double b = mide("hilo uno a uno", hilos_uno_a_uno, N_HILOS);
    double c = mide("proceso en tanda", procesos_tanda, N_PROC);
    double d = mide("proceso uno a uno", procesos_uno_a_uno, N_PROC);
    printf("factor proceso/hilo en tanda : %6.0f x\n", c / a);
    printf("factor proceso/hilo uno a uno : %6.0f x\n", d / b);
    return 0;
}

```

```

$ gcc -O2 -Wall -Wextra -o p2_ej1 p2_ej1.c -lpthread
$ ./p2_ej1

```

Resultado en la máquina de referencia, con Windows y el mejor de 7 repeticiones:

	minimo	media
hilo en tanda :	49.7 us	50.6 us
hilo uno a uno :	86.4 us	89.0 us
proceso en tanda :	2.863 ms	3.198 ms
proceso uno a uno :	17.131 ms	17.951 ms
factor proceso/hilo en tanda :	58 x	
factor proceso/hilo uno a uno :	198 x	

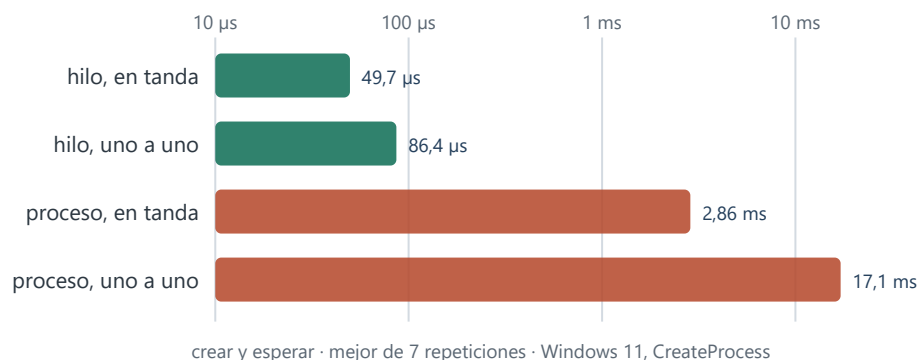


Figura P2.1 · Ejercicio 1. Lo que cuesta crear **una** unidad de ejecución, medido en la máquina de referencia. La escala es logarítmica: cada marca es diez veces más. Un proceso no cuesta «un poco más» que un hilo, cuesta entre 58 y 198 veces más.

Hay dos lecturas, y las dos importan. La primera es el **cociente**: un proceso cuesta entre 58 y 198 hilos. La segunda, y más práctica, es la **cifra absoluta**: un proceso cuesta **milisegundos**. Si tu programa crea procesos dentro de un bucle, cada vuelta arranca con varios milisegundos de peaje que no se recuperan.

Y fíjate en que el factor no es un número: depende de cómo crees. En tanda son 58 veces; de una en una, 198. La razón es que la creación de procesos se solapa mucho mejor entre núcleos que la de hilos, porque cada proceso arranca de forma independiente.

Ahora la misma medida con el programa de este guion en **Linux**, en WSL 2 sobre la misma máquina, en septiembre de 2026:

		minimo	media (5 repeticiones)
hilo	en tanda :	79.6 us	85.6 us
hilo	uno a uno :	86.4 us	91.4 us
proceso	en tanda :	132.0 us	133.8 us
proceso	uno a uno :	248.9 us	267.6 us
factor	proceso/hilo en tanda :	2 x	
factor	proceso/hilo uno a uno :	3 x	

En Linux un proceso cuesta **dos o tres hilos**, no cientos. No es que el ejercicio esté mal hecho: `fork()` no construye nada, duplica el proceso que ya existe y solo copia de verdad la memoria que alguien llegue a escribir, mientras que `CreateProcessA` lo construye todo. La conclusión correcta no es «los procesos son caros», sino que **el coste de un proceso depende del sistema operativo y de cómo lo crees**, y hay que medirlo antes de decidir.

Y una advertencia sobre la variabilidad. Repetido en la misma máquina con Windows en septiembre de 2026, crear un proceso salió a 9,5 ms en tanda y a 37 ms de uno en uno, más del doble que en la tabla, mientras que los hilos cambiaron mucho menos: 54 μ s y 103 μ s. Por eso se publica el mínimo y se dice cuántas repeticiones hay detrás y en qué sistema se midió.

PISTA Comprueba que los hilos te salen del orden de decenas de microsegundos. Si te sale del orden de nanosegundos, el compilador se ha comido algo; si te sale del orden de milisegundos, estás midiendo con un reloj demasiado grueso o el ordenador está muy cargado. En Windows, si el programa dice «no se puede lanzar nada.exe», es que `nada.exe` no está en la carpeta desde la que ejecutas.

Qué va en el PDF

- El código de `p2_ej1.c` tal como lo has ejecutado, pegado como texto.
- La tabla de tus cuatro medidas, con el mínimo y la media, el sistema operativo y el número de repeticiones.
- Dos o tres conclusiones justificadas con tus números: por qué el factor proceso/hilo no es el mismo en tanda que de uno en uno, y cuándo te puedes permitir crear un proceso dentro de un bucle.

Ejercicio 2 (1,5 puntos) — Con qué frecuencia falla una carrera

Enunciado. Compila y ejecuta `p2_ej2.c`: el programa incorrecto más corto que existe — P hilos incrementando un contador global sin proteger— y **mide la probabilidad de que falle** (apartado 2.2.1).

- El programa ejecuta la misma prueba veinte veces por cada casilla y cuenta en cuántas el resultado final es distinto del esperado.
- Además de cuántas fallan, mide **cuánto se pierde**: qué porcentaje de los incrementos se evapora.
- Barre dos ejes: número de hilos (2, 4, 8, 12, 24) y vueltas por hilo (1000, 100 000, 10 000 000).
- Después, **quita el `volatile`** de la declaración del contador, recompila con `-O2` y vuelve a ejecutarlo. Es un cambio de una palabra.
- Tu trabajo es explicar las dos tablas: por qué con pocas vueltas la carrera no aparece, por qué sin `volatile` tampoco, y por qué el programa sigue estando mal en los dos casos.

Este es el corazón del programa. Fíjate en que el error cabe en una línea:

```
/* p2_ej2.c - con que frecuencia falla una carrera (apartado 2.2.1). */
#include <stdio.h>
#include <pthread.h>
```

```

#define PRUEBAS 20

1 static volatile long long contador; /* volatile: obliga a ir a memoria */
  static int vueltas_por_hilo;

  static void *sube(void *arg)
  {
    (void)arg;
    for (int i = 0; i < vueltas_por_hilo; i++)
2      contador++; /* parece una operacion; son tres */
    return NULL;
  }

/* Devuelve el valor final del contador para P hilos y K vueltas cada uno. */
static long long una_ejecucion(int P, int K)
{
  pthread_t h[32];
  contador = 0;
  vueltas_por_hilo = K;
3  for (int i = 0; i < P; i++) pthread_create(&h[i], NULL, sube, NULL);
  for (int i = 0; i < P; i++) pthread_join(h[i], NULL);
  return contador;
}

```

❶ El `volatile` no es un adorno ni un arreglo: está para que la variable viva en **memoria** y no en un registro. Sin él, con `-O2`, cada hilo se lleva su copia al registro y la carrera casi no se ve. Pruébalo en los dos sentidos.

❷ Aquí está todo el error. `contador++` parece una operación y son tres: leer de memoria, sumar uno, escribir. Si dos hilos leen el mismo valor, los dos escriben el mismo valor, y uno de los dos incrementos desaparece.

❸ Se crean los P hilos y se espera a todos antes de mirar el contador, así que el valor final debería ser exactamente $P \times K$.

Y el `main`, que recorre las quince casillas y resume las veinte pruebas de cada una. Los dos fragmentos, uno detrás de otro, forman `p2_ej2.c`:

```

int main(void)
{
  int hilos[] = {2, 4, 8, 12, 24};
  int vueltas[] = {1000, 100000, 10000000};
  for (int k = 0; k < 3; k++) {
    printf("K = %d incrementos por hilo\n", vueltas[k]);
    printf("  hilos    esperado  minimo obtenido  fallan  perdida media\n");
    for (int p = 0; p < 5; p++) {
      long long esperado = (long long)hilos[p] * vueltas[k];
      long long minimo = esperado;
      int fallan = 0;
      double perdida = 0;
      for (int r = 0; r < PRUEBAS; r++) {
        long long v = una_ejecucion(hilos[p], vueltas[k]);
        if (v != esperado) fallan++;
        if (v < minimo) minimo = v;
        perdida += 100.0 * (double)(esperado - v) / (double)esperado;
      }
      printf("  %5d %11lld %17lld  %3d/%d  %10.1f %%\n", hilos[p], esperado,
        minimo, fallan, PRUEBAS, perdida / PRUEBAS);
    }
  }
}

```

```

}
return 0;
}

```

Estos son los resultados en la máquina de referencia. Es un extracto: tu programa imprime las cinco filas de cada K.

K = 1000 incrementos por hilo

hilos	esperado	minimo	obtenido	fallan	perdida media
2	2000		2000	0/20	0.0 %
24	24000		24000	0/20	0.0 %

K = 100000 incrementos por hilo

hilos	esperado	minimo	obtenido	fallan	perdida media
2	200000		200000	0/20	0.0 %
4	400000		107244	6/20	25.7 %
8	800000		243516	11/20	28.0 %
24	2400000		240304	17/20	37.9 %

K = 10000000 incrementos por hilo

hilos	esperado	minimo	obtenido	fallan	perdida media
2	20000000		9019748	20/20	42.8 %
4	40000000		6668127	20/20	74.6 %
8	80000000		4921272	20/20	89.2 %
12	120000000		6465910	20/20	92.8 %
24	240000000		8390216	20/20	95.9 %



con K = 1000 la carrera NO se manifiesta nunca, y el programa es igual de incorrecto

Figura P2.2 · Ejercicio 2. Porcentaje de incrementos que se pierden en un contador compartido sin proteger. Cuanto más dura la ventana de solapamiento, más se pierde: con 24 hilos y diez millones de vueltas cada uno se evapora el 95,9 % del trabajo.

Léelo despacio, porque hay dos lecciones y la segunda es la importante.

La primera: cuando la carrera se manifiesta, **no se pierde un incremento suelto, se pierde casi todo**. Con 24 hilos y diez millones de vueltas cada uno se evapora el 95,9 % del trabajo: de doscientos cuarenta millones de incrementos sobreviven poco más de ocho millones. No es un error de redondeo, es que el programa no hace lo que dice.

La segunda, y esta es la que hay que llevarse: **con mil vueltas por hilo el fallo no aparece jamás**, ni una sola vez en veinte ejecuciones con veinticuatro hilos. Si esa hubiera sido tu prueba, habrías concluido que el programa funciona. El programa es exactamente igual de incorrecto: lo único que pasa es que cada hilo termina antes de que el siguiente llegue a arrancar, así que no hay solapamiento y no hay ocasión de pisarse.

Si trabajas en Linux, con $K = 100\,000$ verás bastante menos fallos. Medido en septiembre de 2026 con este programa en la misma máquina, en WSL fallaron entre 0 y 6 pruebas de cada 20, y en Windows entre 5 y 15. Cuándo llegan a solaparse los hilos depende del planificador del sistema. Lo que no cambia es que el programa es incorrecto.

Y esto es lo que pasa **sin `volatile`**, medido en septiembre de 2026 con este programa y `-O2`, en Windows y en Linux: **0 fallos en las 300 pruebas**, con cualquier número de hilos y de vueltas. El compilador ha convertido el bucle de cada hilo en una sola suma, como cuenta el apartado 2.2.1, así que cada hilo toca la memoria una sola vez y ya casi no hay ocasión de pisarse. El error sigue ahí: es la regla de oro, y la razón de que una prueba que no falla no demuestre nada.

PISTA Si con diez millones de vueltas tampoco te falla, mira dos cosas antes que nada: que el contador sea `volatile`, y que estés compilando con optimización pero sin que el compilador haya podido convertir el bucle en una sola suma. Un truco para comprobarlo: si el tiempo de la versión con 24 hilos es prácticamente el mismo que con 2, es que no hay tráfico de memoria y por tanto no hay nada que se pise.

Qué va en el PDF

- El código de `p2_ej2.c`, pegado como texto.
- Las dos tablas completas, con y sin `volatile`: las tres K por los cinco números de hilos, con ejecuciones fallidas y pérdida media.
- Una **gráfica** de la pérdida frente al número de hilos, con una serie por cada K .
- Dos o tres conclusiones justificadas: por qué con $K = 1000$ no falla, por qué sin `volatile` no falla nunca y qué significa eso para las pruebas de un programa concurrente.

Ejercicio 3 (3,0 puntos) — Tres arreglos, tres precios

Enunciado. Completa y ejecuta `p2_ej3.c`: la misma suma del ejercicio 2, arreglada de las tres formas del apartado 2.2, y **cronometrada**. El guion te da todo el programa salvo **dos funciones, que escribes tú**.

- **Con mutex:** un cerrojo alrededor del incremento (apartado 2.2.2).
- **Con atómico:** `atomic_fetch_add` sobre un `atomic_long`, sin cerrojo (apartado 2.2.5).
- **Con acumulador privado:** cada hilo suma en su variable local y toca la global **una sola vez** al final (apartado 2.2.11).
- La versión sin proteger se mide también, solo como referencia: es incorrecta.
- El programa barre $P = 1, 2, 4, 8, 12, 24$ y comprueba en cada caso que las tres versiones correctas dan el valor exacto.

La primera parte declara los contadores y la versión sin proteger:

```
/* p2_ej3.c - la misma suma, arreglada de tres formas (apartados 2.2.2,
   2.2.5 y 2.2.11), y cronometrada. */
#include <stdio.h>
#include <pthread.h>
#include <stdatomic.h>
#include "crono.h"

#define K    5000000          /* incrementos por hilo */
#define REP 5

static volatile long long c_suelto;          /* INCORRECTA: solo de referencia */
```

```
static long long c_mutex;
static pthread_mutex_t cerrojo = PTHREAD_MUTEX_INITIALIZER;
static atomic_long c_atómico;
static long long c_priv;
static pthread_mutex_t cerrojo_final = PTHREAD_MUTEX_INITIALIZER;

static void *v_suelto(void *a)
{
    (void)a;
    for (int i = 0; i < K; i++) c_suelto++;
    return NULL;
}
```

Lo que escribes tú. Justo debajo de `v_suelto`, dos funciones con su misma forma:

- `static void *v_mutex(void *a)`: hace `K` incrementos de `c_mutex`, tomando `cerrojo` antes de cada incremento y soltándolo justo después (apartado 2.2.2).
- `static void *v_atómico(void *a)`: hace `K` incrementos de `c_atómico` con `atomic_fetch_add`, sin ningún cerrojo (apartado 2.2.5).

Cada una ocupa unas seis líneas. Si te sale mucho más larga, vuelve a la chuleta.

La tercera versión correcta tiene un detalle que hay que entender antes de fiarse de sus números:

```
static void *v_priv(void *a)
{
    (void)a;
    /* volatile A PROPOSITO: sin el, -O2 convierte el bucle en mio = K y se
       mediria nada. Asi escribe en memoria en cada vuelta, como los otros. */
    ① volatile long long mio = 0;                /* local: nadie mas la ve */
    ② for (int i = 0; i < K; i++) mio++;
    pthread_mutex_lock(&cerrojo_final);
    ③ c_priv += mio;                            /* UNA sola vez por hilo */
    pthread_mutex_unlock(&cerrojo_final);
    return NULL;
}
```

① Ese `volatile` está puesto **a propósito y no por corrección**. Sin él, `-O2` ve que `mio` es local y que el bucle solo la incrementa, así que convierte las cinco millones de vueltas en `mio = K` y el tiempo se va a cero. Con `volatile`, este bucle escribe en memoria en cada vuelta igual que los otros tres, y la única diferencia que queda medida es la que interesa: que esa memoria **no la comparte con nadie**.

② Cinco millones de incrementos sobre una variable que solo ve este hilo. Ni cerrojo, ni átomo, ni nada.

③ Y una única operación protegida por hilo, al final. Ahí está toda la ganancia.

Y la última parte cronometra cada versión y comprueba el resultado. Los tres fragmentos del guion, con tus dos funciones en su sitio, forman `p2_ej3.c`:

```
static void r_suelto(void) { c_suelto = 0; }
static void r_mutex(void) { c_mutex = 0; }
static void r_atómico(void) { atomic_store(&c_atómico, 0); }
static void r_priv(void) { c_priv = 0; }

/* Lanza P hilos con la funcion f y devuelve el mejor tiempo de REP. */
static double cronometra(void *(*f)(void *), void (*reinicia)(void), int P)
{
    double mejor = 1e30;
```

```

pthread_t h[32];
for (int r = 0; r < REP; r++) {
    reinicia();
    double t0 = reloj();
    for (int i = 0; i < P; i++) pthread_create(&h[i], NULL, f, NULL);
    for (int i = 0; i < P; i++) pthread_join(h[i], NULL);
    double t = reloj() - t0;
    if (t < mejor) mejor = t;
}
return mejor;
}

int main(void)
{
    int hilos[] = {1, 2, 4, 8, 12, 24};
    printf("  P  suelto(s)  mutex(s)  atomico(s)  privado(s)  correctas  suelto\n");
    for (int k = 0; k < 6; k++) {
        int P = hilos[k];
        long long esperado = (long long)P * K;
        double ts = cronometra(v_suelto, r_suelto, P);
        double tm = cronometra(v_mutex, r_mutex, P);
        double ta = cronometra(v_atómico, r_atómico, P);
        double tp = cronometra(v_priv, r_priv, P);
        int ok = c_mutex == esperado && atomic_load(&c_atómico) == esperado
                && c_priv == esperado;
        printf("%4d %10.4f %9.4f %11.4f %11.4f %-9s %s\n", P, ts, tm, ta, tp,
                ok ? "las 3 si" : "NO", c_suelto == esperado ? "si" : "NO");
    }
    return 0;
}

```

Medido en la máquina de referencia con cinco millones de incrementos por hilo, mejor de 5 repeticiones:

P	suelto(s)	mutex(s)	atomico(s)	privado(s)	correcto?
1	0.0011	0.0448	0.0199	0.0012	suelto:si
2	0.0035	0.1509	0.0595	0.0012	suelto:NO
4	0.0116	0.4619	0.1267	0.0017	suelto:NO
8	0.0490	1.4523	0.3500	0.0026	suelto:NO
12	0.0984	2.3437	0.5335	0.0030	suelto:NO
24	0.2889	4.8179	1.1599	0.0051	suelto:NO

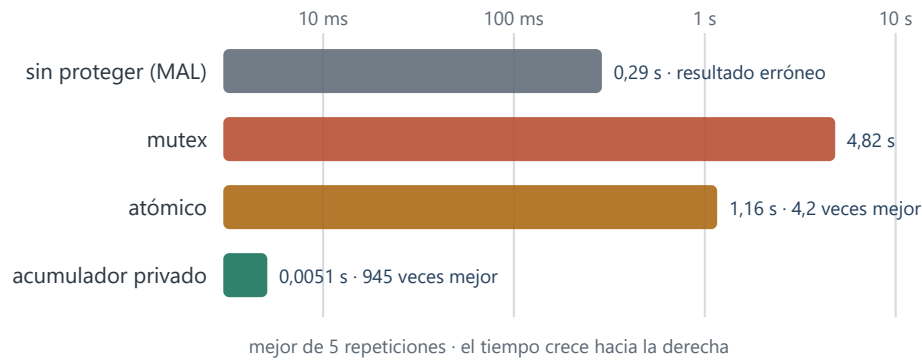


Figura P2.3 · Ejercicio 3. Los tres arreglos de la misma carrera, con 24 hilos y cinco millones de incrementos cada uno. La primera fila está solo como referencia: es rápida porque es incorrecta. Entre las tres correctas hay un factor de **945**.

Conviene traducir la última fila a **coste por incremento**, que es donde se entiende. Con 24 hilos y cinco millones cada uno se han hecho 120 millones de incrementos: el mutex sale a **40 ns** por incremento, el atómico a **9,7 ns** y el acumulador privado a **0,04 ns**. Un factor de **945** entre la peor solución correcta y la mejor.

Y hay algo más importante que el factor. Mira la columna del mutex de arriba abajo: **empeora al añadir hilos**. Con un hilo tarda 0,045 s y con veinticuatro tarda 4,8 s, cien veces más, haciendo cada hilo exactamente el mismo trabajo. Eso es la **contención** del apartado 2.2.3: el cerrojo no reparte el trabajo, lo pone en fila. Un programa así no es que escale mal, es que **escala hacia atrás**.

El acumulador privado, en cambio, sube de 0,0012 a 0,0051 s: se estropea un factor de 4 al pasar de 1 a 24 hilos, que es lo que cuesta la memoria y los núcleos lentos, no la sincronización. Es la única de las tres que se puede llamar escalable, y la razón es que **no sincroniza**.

El programa de este guion imprime la corrección en dos columnas, **correctas** y **suelto**. Vuelto a medir en septiembre de 2026, con 24 hilos salieron 5,5 s el mutex, 1,4 s el atómico y 0,006 s el privado en Windows, y 3,8 s, 1,2 s y 0,005 s en Linux: las cifras bailan, la forma no.

ANTES DE FIARTE DE TUS DOS FUNCIONES Si la columna «correctas» dice NO, el incremento no está **entre** el **lock** y el **unlock**. Y si el mutex te sale casi tan rápido como el atómico, seguramente tomas el cerrojo una sola vez fuera del bucle: el resultado es correcto, pero los hilos ya no compiten por él en cada incremento y no estás midiendo la contención del apartado 2.2.3. Se toma y se suelta en cada vuelta.

PISTA, Y LA LECCIÓN DEL EJERCICIO Si tu tabla sale con el acumulador privado en microsegundos y un factor de miles, sospecha del compilador antes de celebrarlo: puede haber eliminado el bucle. La comprobación se hace con **un solo hilo**: divide el tiempo de la fila $P = 1$ entre sus cinco millones de incrementos. En la tabla de referencia salen $0,0012 \text{ s} / 5\,000\,000 \approx 0,24 \text{ ns}$. Si te sale por debajo de 0,1 ns, ahí no se ha ejecutado ningún bucle: ningún procesador actual hace más de unas pocas operaciones por nanosegundo en un núcleo. No hagas esta comprobación con la fila de 24 hilos dividiendo entre 120 millones: esos hilos trabajan a la vez en núcleos distintos, y la cuenta sale mucho más pequeña sin que nada esté mal.

Qué va en el PDF

- El código de `p2_ej3.c` completo, con tus dos funciones, pegado como texto.
- La tabla completa: los cuatro tiempos para cada P y la comprobación de que las tres versiones correctas dan el valor exacto.
- El coste por incremento de las tres versiones correctas con 24 hilos: el tiempo entre los 120 millones de incrementos.
- Una **gráfica**, mejor en escala logarítmica, con las tres versiones correctas frente a P .
- Dos o tres conclusiones justificadas: por qué el mutex empeora al añadir hilos (apartado 2.2.3), por qué el acumulador privado no, y cuál elegirías en un programa real.

Trabajo autónomo obligatorio: ejercicios 4 y 5 (4 puntos)

Se hacen fuera del aula, con el mismo entorno. Aquí sí hay que **escribir el programa**: cada enunciado da la pieza clave, una pista y la salida de referencia. La estimación es de unas tres horas entre los dos. No necesitan nada que no hayas usado ya en el núcleo: hilos, cerrojos, operaciones atómicas y `crono.h`.

Ejercicio 4 (2,0 puntos) — Construir un interbloqueo y matarlo

Enunciado. Escribe `p2_ej4.c`: dos cuentas bancarias y dos hilos que se hacen transferencias cruzadas, con dos cerrojos y una versión que se cuelga siempre.

- El hilo 0 transfiere de A a B y el hilo 1 de B a A, muchas veces.
- En la versión **mala**, cada hilo bloquea primero la cuenta de origen y después la de destino.
- En la versión **buen**a, los cerrojos se piden siempre en orden creciente de número de cuenta, **sea cual sea el sentido** de la transferencia (apartado 2.2.9).
- Añade un hilo **vigilante** que detecte que nadie avanza y corte la ejecución con un mensaje. Sin él no puedes ejecutar la versión mala más de una vez.
- Ejecuta la versión mala cinco veces y anota en qué transferencia se ha colgado cada vez.
- Comprueba en la versión buena que la suma de los dos saldos es la que era al principio.

El arreglo entero son cuatro líneas:

```
static void transferir(Cuenta *desde, Cuenta *hacia, long long cantidad)
{
    Cuenta *primero = desde, *segundo = hacia;

    /* El arreglo entero cabe en estas dos líneas: si se ordena por id, los
       dos hilos piden los cerrojos en el mismo orden y no hay ciclo. */
    ❶ if (ordenar && primero->id > segundo->id) {
        primero = hacia;
        segundo = desde;
    }

    ❷ pthread_mutex_lock(&primero->cerrojo);
    ❸ pthread_mutex_lock(&segundo->cerrojo);

    desde->saldo -= cantidad;
    hacia->saldo += cantidad;

    pthread_mutex_unlock(&segundo->cerrojo);
    pthread_mutex_unlock(&primero->cerrojo);
}
```

- ❶ Esta condición es todo el arreglo. Si `ordenar` está activo, se intercambian los dos punteros cuando vienen al revés, de modo que `primero` es siempre la cuenta de número más bajo.
- ❷ A partir de aquí los dos hilos piden **el mismo cerrojo primero**, así que uno de los dos gana y el otro espera un instante. No puede haber ciclo.
- ❸ Fíjate en que el segundo `lock` no ha cambiado. Lo que se ha arreglado no es cuántos cerrojos se piden ni durante cuánto tiempo: es **el orden**.

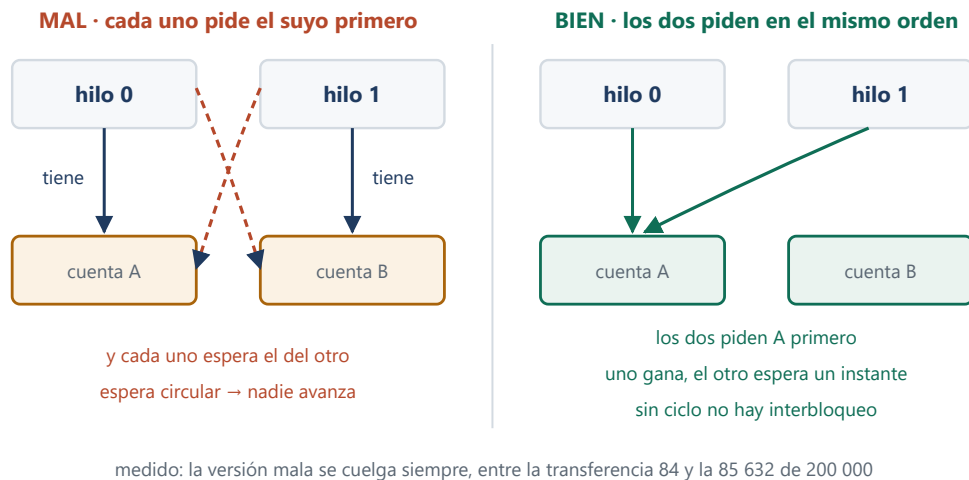


Figura P2.4 · Ejercicio 4. El interbloqueo y su arreglo. La corrección no consiste en poner más cerrojos ni en quitarlos: consiste en pedir siempre los mismos cerrojos **en el mismo orden**.

Y este es el comportamiento observado, cinco ejecuciones de la versión mala:

```
--- cinco ejecuciones de la version MALA ---
*** INTERBLOQUEO *** hilo 0 parado en la transferencia 3517 de 200000
*** INTERBLOQUEO *** hilo 0 parado en la transferencia 1702 de 200000
*** INTERBLOQUEO *** hilo 0 parado en la transferencia 85632 de 200000
*** INTERBLOQUEO *** hilo 0 parado en la transferencia 84 de 200000
*** INTERBLOQUEO *** hilo 0 parado en la transferencia 4556 de 200000

--- version BUENA ---
terminado en 0.0155 s
saldos: A = 1000000 B = 1000000 (suma 2000000, tiene que ser 2000000)
```

Las cinco se cuelgan, sí, pero mira **dónde**: en la transferencia 84 una vez y en la 85 632 otra, con un factor de mil entre ellas. Ese es el rostro real de un interbloqueo en producción. No es un fallo que se dispara al arrancar y se detecta en las pruebas: es uno que puede tardar horas en aparecer, y que aparecerá el día que la máquina esté más cargada.

Repetido en septiembre de 2026 con un programa de referencia, la versión mala se colgó en las cinco ejecuciones tanto en Windows (entre la transferencia 85 y la 2274) como en Linux (entre la 2850 y la 5835), y la buena terminó en 0,015 s y 0,032 s con los saldos exactos.

PISTA El vigilante es más fácil de lo que parece: cada hilo escribe en una variable `atomic_long` —atómica, para no meter una carrera en el propio vigilante— el número de transferencia por la que va, y el vigilante mira esas dos variables cada décima de segundo; para esperarla, `Sleep(100)` en Windows, de `windows.h`, y `usleep(100000)` en Linux, de `unistd.h`. Si en un segundo entero no han cambiado ninguna de las dos, es que están bloqueados y puede cortar. No hace falta ninguna primitiva de detección de interbloqueos ni nada parecido; con eso vale, y de paso te obliga a pensar qué es exactamente «no avanzar».

TRES DETALLES PARA MONTARLO Cada cuenta puede ser `typedef struct { int id; long long saldo; pthread_mutex_t cerrojo; } Cuenta;`, con su cerrojo inicializado con `pthread_mutex_init` al principio del `main`. Para cortar, el vigilante llama a `exit(1)`, de `stdlib.h`: termina el proceso entero aunque los otros dos hilos sigan bloqueados. Y para no recompilar entre versiones, elige la versión con un argumento: `./p2_ej4 mala` o `./p2_ej4 buena`.

Qué va en el PDF

- El código de `p2_ej4.c`, pegado como texto, con las dos versiones y el vigilante.
- La salida de cinco ejecuciones de la versión mala, con el punto donde se cuelga cada una, y la de la versión buena, con el tiempo y la comprobación de los saldos.
- Dos o tres conclusiones justificadas: cuál de las cuatro condiciones de un interbloqueo (apartado 2.2.9) rompe exactamente tu arreglo —las otras tres siguen ahí—, y por qué el punto donde se cuelga cambia tanto entre ejecuciones.

Ejercicio 5 (2,0 puntos) — Un cerrojo, muchos cerrojos o ninguno

Enunciado. Escribe `p2_ej5.c`: un histograma de 256 casillas que rellenan P hilos, en tres versiones todas correctas (apartado 2.2.4).

- **A:** un solo cerrojo global para todo el histograma.
- **B:** un cerrojo por casilla. Solo chocan los hilos que caen en la misma casilla.
- **C:** un histograma privado por hilo, que se suman al final.
- El trabajo **por hilo** es fijo, así que el tiempo debería quedarse clavado al añadir hilos. Publica el factor por el que se estropea, no el speedup: aquí el speedup no significa nada.
- Comprueba en las tres que el total de muestras es exactamente el esperado.

Medido con cuatro millones de muestras por hilo, mejor de 5 repeticiones:

P	global(s)	xN	casilla(s)	xN	privado(s)	xN	ok
1	0.0383	1.00	0.0367	1.00	0.0049	1.00	si
2	0.1298	3.39	0.0976	2.66	0.0050	1.02	si
4	0.4045	10.56	0.1375	3.75	0.0054	1.11	si
8	1.3623	35.56	0.3315	9.03	0.0056	1.15	si
12	2.8282	73.82	0.6761	18.41	0.0077	1.58	si
24	5.8175	151.85	1.2308	33.52	0.0096	1.95	si

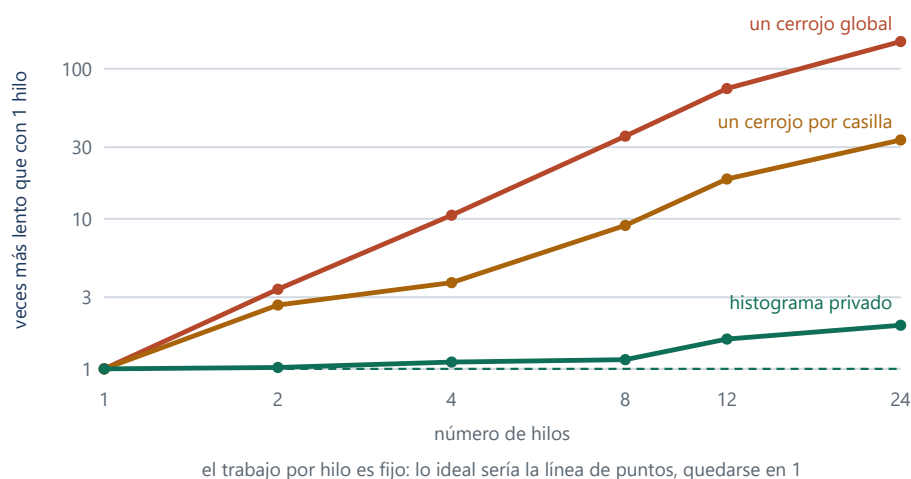


Figura P2.5 · Ejercicio 5. Granularidad del bloqueo, en escala logarítmica. Con un solo cerrojo para todo el histograma, veinticuatro hilos tardan **152 veces** más que uno solo en hacer su parte. Repartir el cerrojo ayuda; no necesitarlo, mucho más.

Con un cerrojo global, veinticuatro hilos tardan **152 veces** más que uno solo en hacer cada uno su parte. Repartir el cerrojo en 256 cerrojos mejora las cosas casi cinco veces, pero sigue siendo un desastre: 33,5. Y no tener cerrojo en el camino caliente cuesta un factor de 1,95, que ya no es sincronización sino memoria y núcleos lentos.

La conclusión que hay que sacar no es «usa cerrojos finos». Es la del apartado 2.2.11: **la mejor sincronización es la que no hace falta**. Casi siempre se puede reorganizar el trabajo para que los hilos no compartan nada que escribir, y cuando se puede, gana por órdenes de magnitud a cualquier refinamiento del bloqueo.

Repetido en septiembre de 2026 con un programa de referencia y 24 hilos, el cerrojo global se estropeó un factor de 157, el de casilla 39 y el privado 2,3 en Windows, y 146, 40 y 3,0 en Linux: las cifras cambian, la forma no.

PISTA Si la versión B no te mejora nada frente a la A, mira dónde están tus 256 cerrojos en memoria. Un `pthread_mutex_t` ocupa bastante menos de una línea de caché, así que varios cerrojos consecutivos caen en la misma línea y los núcleos se pelean por ella aunque estén bloqueando casillas distintas. Es falsa compartición sobre los propios cerrojos, y se arregla igual que cualquier otra: separándolos.

DOS PISTAS MÁS PARA MONTARLO No uses `rand()`. Vas a medir la contención de tus cerrojos, y `rand()` no está pensado para hilos: en Linux lleva un cerrojo dentro, así que estarías midiendo también el suyo sin saber cuál es cuál. Usa un generador propio con el estado en una variable local de cada hilo. Este, de una línea, reparte las 256 casillas de forma uniforme: `static unsigned siguiente(unsigned *s) { *s = *s * 1103515245u + 12345u; return (*s >> 16) & 255u; }`. Dale a cada hilo una semilla distinta, por ejemplo su número más uno. Y para la versión B, declara `pthread_mutex_t cerrojos[256];` e inicializa cada uno con `pthread_mutex_init` en un bucle antes de crear los hilos.

Qué va en el PDF

- El código de `p2_ej5.c` con las tres versiones, pegado como texto.
- La tabla con los tres tiempos y los tres factores de degradación, y la comprobación de que el total de muestras es el esperado.
- Una **gráfica**, mejor logarítmica, con las tres curvas y la línea del comportamiento ideal.
- Dos o tres conclusiones justificadas: por qué la versión B mejora a la A pero no lo suficiente, y qué te dice la C sobre cuándo sincronizar.

Entrega y evaluación

Se entrega **un único PDF por equipo**, llamado `CAR_P2_Apellidos1_Apellidos2.pdf`, con los apellidos de las dos o tres personas del grupo separados por guion bajo. **Solo ese PDF**: no se entregan ficheros de código sueltos, ni carpetas, ni archivos comprimidos, ni enlaces a repositorios. El núcleo se trabaja en clase, y el PDF completo —núcleo y trabajo autónomo— se entrega en la fecha que indique el campus virtual.

No hace falta un informe largo. Dentro del PDF va esto:

- Una **portada** con los nombres completos del equipo, el modelo de procesador, el sistema operativo y el compilador con su versión.
- Para cada ejercicio, **el código pegado como texto**. Copia y pega desde el editor: no valen fotos de la pantalla del código.
- Para cada ejercicio, una **tabla breve** con tus medidas o la captura de la salida del programa, diciendo cuántas repeticiones hay detrás.
- Las **gráficas** de los ejercicios 2, 3 y 5. Sirve cualquier herramienta: una hoja de cálculo basta.
- Para cada ejercicio, **dos o tres conclusiones justificadas** con tus números: las que pide su bloque «Qué va en el PDF».
- Al final, una **declaración del trabajo**: qué ha hecho cada integrante y si habéis usado IA, para qué y qué habéis comprobado vosotros. La IA puede ayudar a escribir o a revisar código; no sustituye a medir ni a entender lo medido, y cada integrante tiene que poder defender cualquier número del PDF.

Cada ejercicio pesa lo que dice su título, y los cinco suman 10:

EJERCICIO	QUÉ SE MIRA SOBRE TODO	PESO
1 · Lo que cuesta nacer	Las cuatro medidas, y distinguir caudal de latencia y Windows de Linux	15 %
2 · Frecuencia de la carrera	Que la carrera se vea, y que se explique el caso en que no aparece	15 %
3 · Tres arreglos	Las tres versiones correctas, el coste por incremento y la explicación de la contención	30 %
4 · Interbloqueo	Que se cuelgue, que se vea colgado y que el arreglo sea un orden total justificado	20 %
5 · Granularidad	Las tres versiones, la métrica correcta y la conclusión sobre no sincronizar	20 %
Total		100 %

Y dentro de cada ejercicio, la nota se reparte así:

CRITERIO	QUÉ SIGNIFICA	PESO
Medida y corrección comprobadas	El programa mide lo que dice medir: reloj adecuado, repeticiones suficientes y resultado comprobado donde hay un valor exacto	30 %
Conclusiones	Se entiende por qué salen esos números. Una tabla sin conclusiones no puntúa	35 %
Presentación de los datos	Tablas legibles, gráficas con ejes rotulados y unidades en su sitio	20 %
Código	Compila sin avisos con <code>-Wall -Wextra</code> ; en el código que escribes tú, además, nombres con sentido y sangrado uniforme	15 %
Total		100 %

Nota de la práctica = ejercicios 1 a 5, hasta 10.

CÓMO SE CORRIGEN LAS CONCLUSIONES No se busca que tus números coincidan con los de este guion. Se busca que sepas defender lo que has medido: qué mediste, con qué reloj, cuántas veces, qué esperabas y por qué salió otra cosa. Un resultado raro bien explicado puntúa más que un resultado limpio sin analizar.

ANTES DE CERRAR EL PORTÁTIL

Cinco minutos que evitan la mitad de los suspensos de esta práctica.

¿Has comprobado el resultado de cada programa, o solo has mirado el tiempo? ¿Puedes demostrar que la carrera del ejercicio 2 se manifiesta, y no solo afirmarlo? ¿La versión rápida del ejercicio 3 hace de verdad sus cinco millones de sumas por hilo? ¿La versión buena del ejercicio 4 pide siempre los cerrojos en el mismo orden, también cuando la transferencia va al revés? Y comprueba que cada tabla lleva al lado cuántas repeticiones hay detrás y en qué sistema se midió.